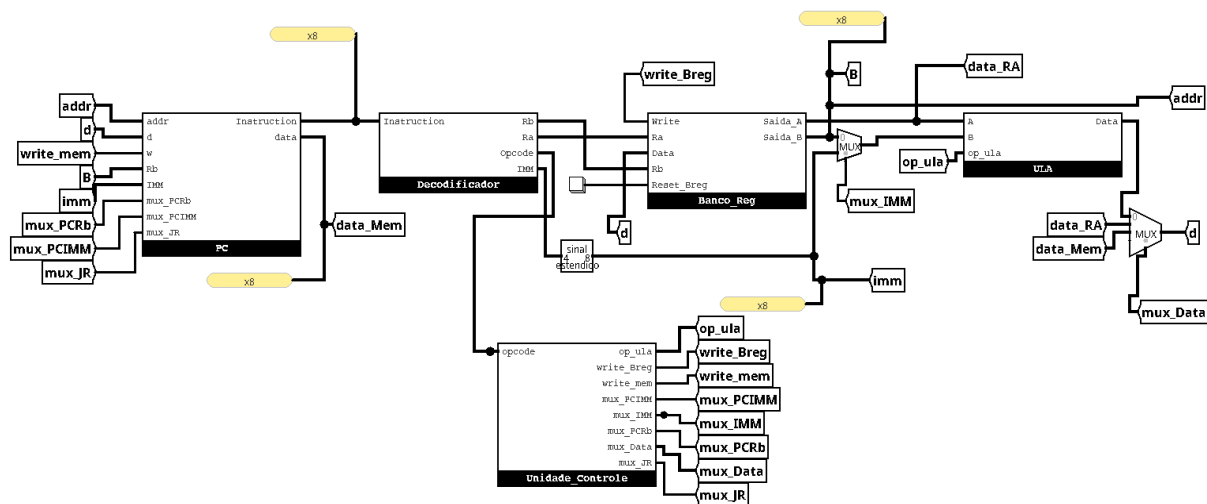


Relatório Trabalho 2 de Arquitetura de Computadores - Redux-V

Por Bruno Bertussi Camillo, GRR 20252947, 26/04/2026

1. Diagrama de Blocos do Datapath



A arquitetura implementada é do tipo monociclo, load-store e segue o modelo Von Neumann, partilhando a mesma memória para instruções e dados através do componente dual port ram. O caminho de dados foi projetado com os seguintes componentes principais:

- **Program Counter (PC):** Registo de 8 bits que armazena o endereço da próxima instrução. Ele possui a Memória RAM (dual port ram), como é Von Neumann, a memória de dados fica junto a memória de instruções.
- **Banco de Registradores:** Contém os 4 registradores (r0 a r3).
- **Unidade Lógica e Aritmética (ULA):** Realiza as operações matemáticas e lógicas.
- **Decodificador:** Decodifica a instrução, quebrando ela nos bits necessários.
- **Unidade de Controle:** Recebe a instrução decodificada e gera os sinais que controlam os multiplexadores, a ULA e as permissões de escrita.

2. Tabela de Códigos da ULA

A ULA foi desenhada para suportar todas as operações aritméticas e lógicas da especificação, bem como as novas instruções. Foi atribuído um código de controlo de 4 bits (op_ula) para cada operação interna:

OP_ULA	Operação
0000	A = 0
0001	not A
0010	A and B
0011	A or B
0100	A xor B
0101	A + B
0110	A - B
0111	A << 2^B
1000	A >> 2^B
1001	A * B
1010	A++

3. Tabela de Controle

A Unidade de Controle principal traduz os 4 bits de Opcode nos seguintes sinais de controlo para o Datapath:

opcode[3..0]	op_ula[3..0]	write_Breg	write_mem	mux_PCIMM	mux_IMM	mux_PCRb	mux_Data[1..0]	mux_JR
0000	0000	0	0	0	0	1	00	0
0001	0010	0	0	1	1	0	00	0
0010	0010	1	0	0	0	0	10	0
0011	0100	0	1	0	0	0	01	0
0100	0101	1	0	0	1	0	00	0
0101	1010	1	0	0	0	0	00	0
0110	1001	1	0	0	0	0	00	0
0111	1001	0	0	0	0	1	00	1
1000	0001	1	0	0	0	0	00	0
1001	0010	1	0	0	0	0	00	0
1010	0011	1	0	0	0	0	00	0
1011	0100	1	0	0	0	0	00	0
1100	0101	1	0	0	0	0	00	0
1101	0110	1	0	0	0	0	00	0
1110	0111	1	0	0	0	0	00	0
1111	1000	1	0	0	0	0	00	0

4. Justificativa para as Instruções Adicionadas

Para otimizar o programa desenvolvido no Trabalho 1 , foram utilizadas as 3 instruções cujos opcodes estavam reservados (0101, 0110, 0111):

1. **inc (Increment) - Opcode 0101:** No código original, o incremento dos iteradores (i++ ou k++) exigia a utilização do r0 com operações addi 1 seguidas de add r1, r0.

Esta instrução permite incrementar diretamente qualquer registo em apenas 1 ciclo de clock, libertando o r0 e limpando a lógica do loop.

2. **mul (Multiplicação) - Opcode 0110:** A arquitetura REDUX-V original não possuía suporte nativo à multiplicação, obrigando o programador a realizar laços de somas sucessivas caso precisasse do produto entre variáveis. Com o mul, o processador é capaz de multiplicar os valores de dois registradores (Ra e Rb) em um único ciclo de clock pela ULA.
3. **jr (Jump Register) - Opcode 0111:** A arquitetura sofria com uma forte limitação de distância: a instrução nativa ji (Jump Immediate) permitia saltar apenas 7 instruções de cada vez, devido ao tamanho do seu operando imediato. Isto tornava a navegação em programas maiores muito difíceis. Com o jr, basta guardar o endereço de destino num registo e instruir o Program Counter (PC) a assumir esse valor. Isto eliminou a restrição espacial, permitindo saltar para qualquer parte da memória num único ciclo.

5. Código Assembly Otimizado do Novo Programa

```
xor r1, r1 ; r1 (i) = 0
xor r2, r2 ; r2 (k) = 0
xor r3, r3 ; início do loop 1
xor r0, r0
addi 5 ; r0 = 5
add r0, r0 ; r0 = 5 + 5 = 10
sub r0, r1 ; r0 = 10 - i
add r3, r0 ; r3 = 10 - i
xor r0, r0 ; r0 = 0
addi 5
mul r0, r0
addi 4 ; r0 = Endereço fim loop 1
brzr r3, r0
xor r0, r0
addi 5
addi 3
mul r0, r0
addi -1 ; r0 = 63
add r0, r1 ; r0 = 63 + i = &A[i]
st r2, r0 ; A[i] = k
inc r2 ; k++
addi 5
addi 5 ; &A[i] + 10 = &B[i]
st r2, r0 ; B[i] = k
inc r2 ; k++
inc r1 ; i++
xor r0, r0
addi 2 ; r0 = 2
jr r0 ; volta pro inicio do loop
xor r1, r1
```

```

xor r3, r3 ; início do loop 2
xor r0, r0
addi 5
add r0, r0
sub r0, r1
add r3, r0 ; r3 = 10 - i
xor r0, r0
addi 5
addi 3
mul r0, r0
addi -2 ; r0 = Endereço fim loop 2 = 62
brzr r3, r0
xor r0, r0
addi 5
addi 3
mul r0, r0
addi -1 ; r0 = 63
add r0, r1 ; r0 = 60 + i = &A[i]
ld r2, r0 ; r2 = M[&A[i]]
addi 5
addi 5 ; r0 = &B[i]
ld r3, r0 ; r3 = M[&B[i]]
addi 5
addi 5 ; r0 = &R[i]
add r2, r3 ; r2 = A[i] + B[i]
st r2, r0 ; M[&R[i]] = A[i] + B[i]
inc r1 ; i++
xor r0, r0
addi 5
mul r0, r0
addi 5 ; r0 = 30
jr r0 ; volta para o inicio do loop 2
ji 0 ; equivalente ao ebreak

```